

📄 Résumé de la faille

Le développeur a laissé une **porte dérobée** dans le code : un commentaire caché (encodé en ROT13) révélait qu'envoyer l'en-tête HTTP `X-Dev-Access: yes` contourne l'authentification. En forgeant une requête de login avec cet en-tête (via `curl`), on accède au portail **sans connaître le mot de passe** → flag.

Faille : backdoor développeur / bypass d'authentification par en-tête HTTP oublié en production.

⚙️ Méthodo — les étapes

1. Lire l'énoncé + les hints - « le développeur a laissé une entrée secrète » - Hint : *les devs laissent des notes dans le code, pas toujours en clair* - Hint : rotation de 13 lettres → **ROT13**

2. Lire le code source de la page (Ctrl+U) - Trouvé un commentaire HTML non lisible :

```
<!-- ABGR: Wnpx - grzcbenel olcnff: hfr urngre "K-Qri-Npprrff: lrf" -->
<!-- Remove before pushing to production! -->
```

3. Décoder le commentaire (CyberChef → ROT13) - Résultat :

```
NOTE: Jack - temporary bypass: use header "X-Dev-Access: yes"
```

- il existe un en-tête secret qui contourne le login.

4. Analyser le mécanisme de login (dans le code JS) - Le formulaire envoie un POST vers `/login` en JSON :

```
fetch('/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password })
})
```

- Problème** : le formulaire n'ajoute jamais l'en-tête `X-Dev-Access`. Et on ne peut pas ajouter un en-tête HTTP via un champ de formulaire.

5. Forger la requête à la main (curl)

```
curl -X POST http://[URL]:PORT/login \
-H "Content-Type: application/json" \
-H "X-Dev-Access: yes" \
-d '{"email":"ctf-player@picoctf.org","password":"x"}'
```

- Le serveur voit l'en-tête magique → ignore le mot de passe → répond :

```
{"success": true, "flag": "picoCTF{...}"}
```

🤖 Pourquoi ça marche

- Le serveur a une logique du type : *si l'en-tête `X-Dev-Access: yes` est présent → accès accordé, on ne vérifie pas le mot de passe.*
- Le dev avait mis ça pour tester pendant le développement, et a oublié de l'enlever (`Remove before pushing to production!`).
- Le **mot de passe bide**n (`x`) **se** passe parce que la backdoor l'ignore complètement.
- Nom du flag `brut4_f0rc4` = clin d'œil : le brute force était inutile, il fallait trouver la backdoor.

🔑 Le concept clé : forger ses propres requêtes

Le formulaire / navigateur ne contrôle **PAS** ce qu'on envoie vraiment au serveur.

Le formulaire n'est qu'une façade. Une requête HTTP, c'est du texte qu'on peut fabriquer soi-même, avec les en-têtes qu'on veut. Le formulaire n'ajoutait pas `X-Dev-Access` → on le fait nous-même avec curl.

On n'est **jamais** limité par l'interface web : curl, la console du navigateur, ou Burp Suite permettent d'envoyer n'importe quelle requête custom.

🧰 Outils utilisés

Outil	Usage
DevTools / Ctrl+U	Lire le code source, trouver le commentaire caché
CyberChef (ROT13)	Décoder le commentaire
curl	Forger la requête POST avec l'en-tête custom

Alternatives à curl pour ce challenge : - Console navigateur : réutiliser `fetch('/login', {...})` en ajoutant l'en-tête - Burp Suite : intercepter la requête et ajouter l'en-tête (méthode pro)

🔗 Réflexes à retenir

- **Lire le code source** (Ctrl+U) : les commentaires cachent souvent des indices.
- **Décoder ce qui a l'air encodé** : ROT13, base64... (CyberChef "Magic" détecte tout seul).
- **Les en-têtes HTTP sont une surface d'attaque** : en-têtes custom (`X-...`), User-Agent, etc. peuvent cacher des bypass.
- **Forger ses requêtes** (curl/Burp/console) dès que le formulaire ne permet pas ce qu'on veut.
- Une **backdoor dev oubliée en prod** est une faille réelle et fréquente.

🔗 Rappel curl (options utilisées)

Option	Rôle
<code>-X POST</code>	Méthode HTTP (POST pour envoyer des données)
<code>-H "Nom: Valeur"</code>	Ajoute un en-tête (autant de <code>-H</code> que voulu)
<code>-d '...'</code>	Corps de la requête (les données envoyées)
<code>-v</code>	Verbose : voir tous les détails requête/réponse
<code>-i</code>	Inclure les en-têtes de la réponse
<code>-b "cookie=val"</code>	Envoyer un cookie (rejouer une session)
<code>-L</code>	Suivre les redirections